Dokumentasi Troubleshooting NRF24

Dokumentasi Troubleshooting NRF24 — Transmitter "selalu FAILED"

Dokumen ini merangkum kenapa pengiriman dari transmitter ([receiverArduino.py](receiverArduino.py — di folder `Master Code/Transmitter/`)) **selalu balik FAILED**, dan cara-cara mendiagnosa/memperbaikinya.

---

1. Apa arti "FAILED"?

Di transmitter:

```
result = radio.write(payload.encode())
if result: print("SUCCESS")
else:      print("FAILED")
```

`radio.write()` balik:

- **SUCCESS** → paket dikirim **DAN** transmitter menerima *auto-ACK* balik dari receiver.
- **FAILED** → paket dikirim tapi **tidak ada auto-ACK** dalam batas retry.

Jadi **FAILED bukan berarti sinyal TX-nya rusak.** Paketnya kemungkinan besar terpancar, tapi ESP32 receiver **tidak menjawab (ACK)** saat itu. Masalahnya ada di sisi receiver tidak dalam kondisi *listening/ACK*, bukan di transmitter.

> **Insight kunci:** Output transmitter itu sendiri adalah alat diagnosa utama. Begitu `write()` mulai balik **SUCCESS**, itu bukti receiver sudah hidup & listening.

---

2. Kondisi saat ini (yang sudah dites)

- Tes baling-baling (ESC/motor) → **aman**.
- Tes radio (terpisah) → **aman**.
- Digabung di `Finalnew.ino` + kirim dari transmitter → **selalu FAILED**.

Pola "masing-masing aman, digabung gagal" ini mengarah ke **interaksi antar subsistem** (urutan boot, atau power), bukan salah satu komponen yang rusak.

3. Konfigurasi radio — sudah COCOK di kedua sisi

Parameter	Transmitter (`receiverArduino.py`)	Receiver (`Finalnew.ino`)
Address	`b"00001"` (TX pipe)	`"00001"` (reading pipe 1)
Channel	`0x76`	`0x76`
Data rate	`RF24_250KBPS`	`RF24_250KBPS`
PA Level	`RF24_PA_LOW`	`RF24_PA_LOW`

Karena config sudah sama, penyebabnya **bukan salah setting**, melainkan salah satu hipotesis di bawah.

---

4. Hipotesis penyebab (urut dari paling mungkin)

#1 — Urutan `setup()` : NRF diinit paling akhir ⟵ paling curiga

Di `Finalnew.ino` fungsi `setup()` (sekitar baris 184–223), NRF baru diinit **setelah**:

1. `mpu.calcGyroOffsets(true)` — makan ~3 detik, dan **bisa nge-hang selamanya** kalau MPU6050 tidak terbaca di I2C.
2. `armESC()` → `delay(5000)` — 5 detik.

Efeknya `radio.startListening()` baru jalan **±8 detik setelah boot** — atau **tidak pernah** kalau MPU nyangkut. Selama itu receiver tidak nge-ACK → transmitter FAILED.

Cek cepat: kalau lewat Serial Monitor (115200) tidak pernah muncul `Receiver Ready`, atau nyangkut di "Arming ESC", ini penyebabnya.

#2 — Brownout power NRF saat ESC/motor aktif

NRF24 sangat sensitif ke supply 3.3V. Saat motor menarik arus, tegangan drop → NRF gagal ACK. Ini penjelasan klasik "props aman, radio aman, digabung gagal".

Fix: solder kapasitor **10µF (elco) + 100nF** langsung di pin VCC–GND modul NRF, dan pastikan NRF disuplai 3.3V stabil (idealnya regulator/adapter NRF sendiri, bukan menumpang 3.3V ESP32 yang sudah terbebani).

#3 — Modul NRF clone tidak stabil di 250KBPS

Banyak modul NRF24 KW tidak reliable di `RF24_250KBPS`.

Fix: turunkan ke `RF24_1MBPS` di **kedua sisi**:

- Python: `radio.setDataRate(RF24_1MBPS)`
- `Finalnew.ino` : `radio.setDataRate(RF24_1MBPS)`

---

5. Cara diagnosa / atasi (menu opsi)

A. Reorder `setup()` — pindah init NRF ke paling awal ⟵ rekomendasi (ini FIX-nya)

Init NRF (`radio.begin()` → `startListening()`) dipindah ke **paling atas** `setup()`, sebelum ESC & MPU. Radio langsung listening; MPU/ESC yang lambat atau nge-hang **tidak lagi memblok** radio. Langsung menghapus penyebab #1.

- Effort: kecil (cuma pindah urutan blok). Hardware tambahan: tidak ada.

B. ACK status payload — pantau dari sisi transmitter (tanpa lihat robot)

Receiver menempelkan status singkat (mis. `"RDY"`) di auto-ACK; transmitter membacanya. Kamu cukup lihat layar Pi/laptop transmitter — **tidak perlu menyentuh robot**.

- Receiver (ESP32):

```
radio.enableAckPayload();
radio.enableDynamicPayloads();
// saat siap / tiap loop:
radio.writeAckPayload(1, "RDY", 3);
```

- Transmitter (pyrf24):

```
radio.enableDynamicPayloads()
radio.enableAckPayload()
if radio.write(payload.encode()):
    if radio.available():
        status = radio.read(radio.getDynamicPayloadSize())
        print("STATUS:", status)
```

- Effort: sedang. Hardware tambahan: tidak ada. Bagus digabung dengan A.

C. Buzzer — indikator suara di lapangan

Beep sebagai penanda status, kedengeran dari jarak jauh di tambak: 1 beep = radio ready, beep cepat terus = NRF fail, beep tiap paket masuk.

- Effort: kecil–sedang. Hardware: buzzer + 1 pin GPIO bebas.

D. Telemetry nirkabel (WiFi / Bluetooth Serial ESP32)

ESP32 kirim log status via `BluetoothSerial` atau halaman web WiFi, dibaca dari HP/laptop tanpa kabel.

- Effort: besar (kode paling banyak, bisa ganggu timing NRF). Hardware: tidak ada.

E. Tanpa ubah kode — akali USB

Pakai kabel USB extension, atau colok & tes di meja **sebelum** robot dirakit; buka Serial Monitor (115200) dan pastikan muncul `Receiver Ready`.

- Effort: nol perubahan kode. Hardware: kabel USB panjang.

---

6. Tabel perbandingan

Cara	Effort	Butuh hardware	Dipantau dari	Sekaligus fix?
A. Reorder setup	Kecil	–	(fix, tak perlu dipantau)	✅ ya (#1)
B. ACK status payload	Sedang	–	layar transmitter	Diagnosa
C. Buzzer	Kecil	Buzzer + 1 pin	suara di lapangan	Diagnosa
D. Telemetry WiFi/BT	Besar	–	HP/laptop nirkabel	Diagnosa
E. Tanpa ubah kode	Nol	Kabel USB panjang	Serial Monitor	Diagnosa

---

7. Rekomendasi

1. **Terapkan A (reorder setup)** — ini memperbaiki penyebab paling mungkin (#1).
2. Opsional tambah **B (ACK status)** supaya bisa memantau kondisi receiver langsung dari transmitter tanpa menyentuh robot.
3. Kalau setelah A masih FAILED → lanjut ke **#2 (kapasitor power NRF)** lalu **#3 (turunkan ke 1MBPS di dua sisi)**.

---

8. Langkah verifikasi

1. Upload `Finalnew.ino` ke ESP32, nyalakan robot.
2. Jalankan `receiverArduino.py` di transmitter.
3. Masukkan misi, mis. `P=10; L=5; MODE=AUTO`.
4. **Berhasil** kalau transmitter mencetak **SUCCESS** (opsi B: plus status `RDY`).
5. Kalau masih **FAILED**:
    - Pastikan (via Serial / buzzer / ACK) receiver benar-benar mencapai kondisi *listening*.
    - Cek power NRF (kapasitor) dan coba data rate `1MBPS` di kedua sisi.